

COMPUTER SCIENCE AND ENGINEERING, IOANNINA

27/05/2026

NATURAL LANGUAGE PROCESSING (NLP)

Food hazard detection ST1 Challenge

ELENI PAPAZOGLOU 5323

AGLAIA LIANOU 5268

Kaggle team name: 'we're going up up up'

Food Hazard Detection ST1 Challenge

Table of Contents

1. Introduction	2
2. Dataset Analysis.....	2
3. Input Creation and Labels.....	3
4. Experimental Workflow.....	3
5. Experiments and Results	3
6. Final Selected Model	12
7. Kaggle Final Evaluation.....	14
8. Limitations	16
9. Computational Setup.....	16

Food Hazard Detection ST1 Challenge

1. Introduction

In this project, we worked on the **Food Hazard Detection ST1 Challenge**. The task was to predict two labels for each food recall incident: **hazard-category** and **product-category**. Our goal was to train models on the training set, evaluate them on the validation set using the official metric, and finally create predictions for the test set to submit on Kaggle.

2. Dataset Analysis

The dataset was split into three files: train.csv, valid.csv, and test.csv. The train and validation sets contained both the input data and the correct labels, while the test set contained only the input data and was used for the final Kaggle submission.

The three datasets contained different numbers of examples:

Datasets	Number of examples
train.csv	5,082
valid.csv	565
test.csv	997

The first step was to load the three provided datasets using pandas. Then, we checked the shape of each dataset, the available columns, and whether there were any missing values. Although no missing values were found, we still used safe handling for empty values during input creation to make the preprocessing pipeline more robust. We also confirmed that the dataset had strong class imbalance, since some categories appeared much more frequently than others.

The distribution of the hazard-category labels in the training set was:

HAZARD-CATEGORY	COUNT
allergens	1854
biological	1741
foreign bodies	561
fraud	371
chemical	287
other hazard	134
packaging defect	54
organoleptic aspects	53
food additives and flavouring	24
migration	3

A similar imbalance was also observed in the product-category label, where some product types appeared very often, while others had only a few examples.

This imbalance was important because it made the task harder, especially for the rare classes.

3. Input Creation and Labels

After the initial data analysis, we created a new column called ``input_text``. This column combined the ``title`` and ``text`` fields into one text input for each example. Then, we separated the input text from the target labels. The ``input_text`` column was used as the input for the models, while ``hazard-category`` and ``product-category`` were used as the two **target labels**. Since the test set did not contain labels, it was only used later to generate the final Kaggle predictions.

4. Experimental Workflow

After creating the input text and separating the labels, we followed a standard text classification pipeline. First, the textual data were preprocessed and converted into numerical features. Then, different machine learning models were trained to predict the two target labels: hazard-category and product-category. Finally, the predictions on the validation set were compared with the true validation labels using the official metric.

Although the two labels were predicted with separate classifiers, the official metric connects them because product-category is evaluated only when hazard-category is predicted correctly. The official score is calculated as:

Official score =

(macro-F1 hazard-category + macro-F1 product-category when hazard is correct) / 2

This means that the hazard-category prediction is very important.

The experiments began with simple classical machine learning baselines and gradually moved to more advanced feature representations and hybrid model setups.

5. Experiments and Results

Experiment 1: TF-IDF + Logistic Regression

TF-IDF was used to convert the text into numerical vectors. After this step, each recall text was represented as a row in a large matrix, where each column corresponded to a word or term from the vocabulary. The TF-IDF value showed how important each term was for a specific text compared to the whole dataset. These numerical vectors were then given as input to Logistic Regression which is a linear classifier that learns a weight for each feature. During training, the model learns which words or terms are strongly connected with each class. For example, some terms may increase the probability of the class allergens, while others may be more useful for biological or foreign bodies. We trained two separate Logistic Regression models: one for ``hazard-category`` and one for ``product-category``.

The results:

METRIC	SCORE
F1 hazard	0.7019

Food Hazard Detection ST1 Challenge

F1 product when hazard correct	0.5000
Official score	0.6009
Kaggle score	0.6046

The results showed that this simple baseline was not strong enough, especially for the `product-category` prediction. However, it was useful as a starting point for the rest of the experiments.

Experiment 2: TF-IDF + LinearSVC

We replaced Logistic Regression with LinearSVC, which is a linear Support Vector Machine classifier. It tries to find decision boundaries that separate the classes as well as possible. This type of model often works well with TF-IDF features, because TF-IDF creates high-dimensional sparse vectors. We used $C = 0.75$. The parameter C controls the regularization strength of the model. A smaller value, such as $C = 0.20$, forces the model to be simpler and reduces overfitting, but it may underfit. A larger value, such as $C = 1$, allows the model to fit the training data more closely, but it can increase the risk of overfitting.

The results:

METRIC	SCORE
F1 hazard	0.7920
F1 product when hazard correct	0.6842
Official score	0.7381
Kaggle score	Not submitted

The score improved a lot compared to the Logistic Regression baseline. The official validation score increased from 0.6009 to 0.7381, showing that LinearSVC was a better classifier for this TF-IDF representation. This version was not submitted to Kaggle, because we first wanted to keep LinearSVC and further improve the TF-IDF representation and test stronger feature settings.

Experiment 3: Word + Char TF-IDF + LinearSVC

We kept LinearSVC as the classifier, but improved the TF-IDF representation. Instead of using only word-level TF-IDF features, we combined word TF-IDF with character TF-IDF.

The word-level TF-IDF used unigrams and bigrams, with `ngram\_range=(1,2)`, while the character-level TF-IDF used character n-grams from 3 to 5 characters, with `ngram\_range=(3,5)`. This allowed the model to use both whole-word information and smaller character patterns.

The results:

METRIC	SCORE
F1 hazard	0.8056

Food Hazard Detection ST1 Challenge

F1 product when hazard correct	0.7161
Official score	0.7608
Kaggle score	0.74841

This experiment improved the official validation score from **0.7381** to **0.7609**. The Kaggle score also improved, showing that adding character-level TF-IDF features made the model stronger.

Experiment 4: Double Title + Better C Values + Word + Char TF-IDF + LinearSVC

We kept the Word + Character TF-IDF representation and the LinearSVC classifiers. We repeated the title field in the input text, because it usually contains the most important information, and tuned the C values using a small validation script. The script found that C = 1 for hazard-category and C = 1.25 for product-category was the best combination.

We also tested simple text cleaning, but it made almost no difference, which suggests that the dataset was already clean enough for TF-IDF models.

The results:

METRIC	SCORE
F1 hazard	0.8123
F1 product when hazard correct	0.7248
Official score	0.7685
Kaggle score	0.7498

This experiment improved the official validation score from 0.7609 to 0.7685. The improvement was mainly caused by giving more weight to the title and by tuning the C values.

Experiment 5+6: Feature Selection and Dimensionality Reduction

After the best LinearSVC setup, we tested two extra methods on top of the Word + Character TF-IDF representation (Exp 4).

In **Experiment 5**, we first created the **Word + Character TF-IDF features** and then applied **Chi-square feature selection**. This means that after TF-IDF created a large feature matrix, Chi-square tried to keep only the features that were most related to the target classes. However, this did not improve the score.

In **Experiment 6**, we again started from the **Word + Character TF-IDF features**, but this time we applied **TruncatedSVD/LSI** which is a dimensionality reduction method. It takes the large TF-IDF matrix and compresses it into fewer dimensions. The goal was to reduce the size of the feature space and keep the most important information. However, this approach performed much worse and the official score dropped. This probably happened

Food Hazard Detection ST1 Challenge

because the compression removed useful rare words and character n-grams, which were important for detecting specific hazards and product categories.

The results:

EXPERIMENT	METHOD	F1 HAZARD	F1 PRODUCT	OFFICIAL SCORE
5	Word + Char TF-IDF + Chi-square + LinearSVC	0.8056	0.7161	0.7608
6	Word + Char TF-IDF + TruncatedSVD/LSI + LinearSVC	0.7672	0.4925	0.6298

The results showed that neither method improved the previous best setup. So, we decided not to use feature selection or dimensionality reduction in the next experiments and continued with the full Word + Character TF-IDF representation.

Experiment 7: FastText TF-IDF Weighted Embeddings + LinearSVC

In this experiment, we tested a different text representation based on FastText embeddings. FastText represents words as dense numerical vectors and can also capture subword information, which can be useful for words with similar forms.

To create one vector for each recall text, we combined the FastText word vectors using TF-IDF weights. This means that more important words in a text had a larger effect on the final text representation. Then, we trained LinearSVC classifiers using these embedding-based features.

The results:

METRIC	SCORE
F1 hazard	0.5718
F1 product when hazard correct	0.4540
Official score	0.5129
Kaggle score	Not submitted

This showed that the model was probably predicting the frequent classes well, but it struggled with rare classes. Since the official metric is based on macro-F1 and not accuracy, this approach was much worse than the TF-IDF models.

Experiment 8: Structured Input + Word + Char TF-IDF + LinearSVC

Food Hazard Detection ST1 Challenge

In this experiment, we returned to the Word + Character TF-IDF representation (Exp 4) and LinearSVC classifiers, but changed the input format. Instead of using only the simple `title + text` input, we created a more structured input using product, problem, and description information.

The goal was to give the model a clearer text structure and help it separate information related to the product from information related to the hazard.

The results:

METRIC	SCORE
F1 hazard	0.8059
F1 product when hazard correct	0.7213
Official score	0.7636
Kaggle score	Not submitted

This approach performed much better than the FastText embedding experiment and was close to the best LinearSVC setup. However, it did not improve over Experiment 4, so we did not keep this input format as the final approach.

Experiment 9: SGDClassifier with Hinge Loss

After the LinearSVC experiments, we tested SGDClassifier as an alternative linear classifier. SGDClassifier is trained with stochastic gradient descent. Instead of learning the model in one large optimization step, it updates the model weights gradually using the training examples. This makes it efficient for large and sparse feature spaces, such as TF-IDF text features.

We tested different combinations of loss and alpha values for the hazard and product classifiers using a small tuning script. The script tested different settings separately for hazard-category and product-category, because the best parameters for the two labels were not necessarily the same.

The best combination was loss="hinge" for both classifiers, alpha = 3e-5 for hazard-category, and alpha = 1e-4 for product-category. The hinge loss makes SGDClassifier work similarly to a linear SVM, while alpha controls regularization.

The results:

METRIC	SCORE
F1 hazard	0.8377
F1 product when hazard correct	0.7178
Official score	0.7778
Kaggle score	0.7566

This experiment improved the previous best validation score, mainly because the hazard F1 increased significantly. Since the official metric depends strongly on correct hazard predictions, this made SGDClassifier an important model for the next experiments.

Food Hazard Detection ST1 Challenge

Experiment 10: Hybrid SGDClassifier + LinearSVC

After the full SGDClassifier setup, we tested a hybrid model. We kept SGDClassifier for hazard-category, because it improved hazard F1, but used LinearSVC for product-category, because it had performed well in earlier experiments.

We again used a double-title input format: title + title + text. The hazard model was SGDClassifier(loss="hinge", alpha=3e-5, class_weight="balanced"), while the product model was LinearSVC(C=1.25, class_weight="balanced").

The results:

METRIC	SCORE
F1 hazard	0.8377
F1 product when hazard correct	0.7191
Official score	0.7784
Kaggle score	0.76591

This setup gave a better Kaggle score than the previous full-SGD model, so we continued with the hybrid approach.

Experiment 11: Separate TF-IDF Settings for Hazard and Product

We continued from the hybrid setup and tested separate TF-IDF settings for hazard-category and product-category. The idea was that the two labels may benefit from different n-gram patterns. After testing different settings, we kept the stronger hazard setup and changed only the product TF-IDF settings.

The best version used word n-grams (1,2) and character n-grams (3,5) for hazard, while product used word n-grams (1,3) and character n-grams (3,5).

The results:

METRIC	SCORE
F1 hazard	0.8377
F1 product when hazard correct	0.7273
Official score	0.7825
Kaggle score	0.7664

Experiment 12: Hazard Tuning with Fixed Product Model

We kept the best product pipeline from Experiment 11 fixed and focused only on improving the hazard-category classifier. We tested different TF-IDF settings and SGDClassifier parameters for the hazard model. The best setup used word n-grams (1,2), character n-grams (3,6), and SGDClassifier with loss="hinge" and alpha = 4e-5.

The results:

Food Hazard Detection ST1 Challenge

METRIC	SCORE
Official score	0.7912
Kaggle score	0.76890

This improved both the validation score and the Kaggle score compared to Experiment 11, confirming that the hazard classifier was very important for the final metric.

Experiment 13: Hazard Tuning with Fixed Product Model

We kept the setup from Experiment 12 and tested different input formats. More specifically, we tried versions with repeated titles 2 and 3 times and versions that also included metadata, such as country, year, and month. The goal was to check if giving the model more structured information would improve the predictions.

The results:

METRIC	SCORE
F1 hazard	0.8522
F1 product when hazard correct	0.7301
Official score	0.7912
Kaggle score	0.7559

Although this setup slightly improved the validation hazard and product F1 scores, the Kaggle score decreased. This suggests that the changes may have overfitted the validation set.

Experiment 14: Product TF-IDF with Accent Normalization

In this experiment, we kept the same general setup as before, but made a small change in the product TF-IDF preprocessing. We added ``strip_accents="unicode"`` to the word and character TF-IDF vectorizers.

The goal was to normalize accented and Unicode characters before creating the TF-IDF features. This helps the vectorizer treat similar words more consistently and avoid creating different features only because of accent or encoding differences.

The results:

METRIC	SCORE
F1 hazard	0.8522
F1 product when hazard correct	0.7331
Official score	0.7926
Kaggle score	0.76876

This change slightly improved the product score and also gave a better Kaggle score.

Experiment 15: Classical Grid Search

Food Hazard Detection ST1 Challenge

In this experiment, we combined the best ideas from the previous classical experiments and tested them more systematically. We tried different input formats, TF-IDF settings, and classifier parameters for both hazard-category and product-category.

For hazard-category, we used SGDClassifier with Word + Character TF-IDF. For product-category, we used LinearSVC with tuned TF-IDF settings and C values. The best setup from this stage used title2_text_meta for hazard and title3_text_meta for product.

The results:

METRIC	SCORE
F1 hazard	0.8582
F1 product when hazard correct	0.7480
Official score	0.8031
Kaggle score	0.77507

This experiment confirmed that separate optimized pipelines for the two labels were more effective than using one common setup for both tasks.

Experiment 16-19: Neural, Embedding and Auxiliary Experiments

After the classical TF-IDF experiments, we also tested heavier and more complex approaches. The goal was to check whether embeddings, transformer-based models, or auxiliary fine-label information could improve the results beyond the tuned classical pipelines.

These experiments were more expensive to train and did not always generalize well to the Kaggle test set. For this reason, we present them separately from the main classical TF-IDF experiments.

The results:

Experiment	Method	F1 Hazard	F1 product when hazard correct	Official score	Kaggle score
16	RoBERTa-base for both labels	0.8755	0.7083	0.7919	0.68984
17	RoBERTa hazard + SVC product	0.8430	0.6799	0.7614	0.73277
18	RoBERTa-large hazard + TF-IDF product	0.7592	0.6761	0.7177	0.73188
19	SBERT embeddings + XGBoost	0.5196	0.5773	0.5485	Not submitted

In these experiments, we wanted to see if heavier models could beat the TF-IDF pipelines. For the RoBERTa experiments, the input text was not converted into TF-IDF features. Instead, it was passed through the RoBERTa tokenizer, which splits the text into smaller tokens. Then, the RoBERTa model processed these tokens and a classification layer predicted the final label.

Food Hazard Detection ST1 Challenge

We tried RoBERTa in different ways. First, we trained RoBERTa-base for both labels. This gave a high validation hazard F1, but the Kaggle score was much lower. Then, we tried using RoBERTa only for the hazard label and keeping an SVC-based model for the product label, but this also did not improve the final result. We also tested RoBERTa-large for hazard, but it performed worse and was much heavier to train.

We also tested SBERT embeddings with XGBoost. In this case, each text was first converted into one dense embedding vector using SBERT, and then XGBoost used these vectors to classify the examples. This approach performed much worse than the TF-IDF models.

These experiments were not developed further, because their scores were not close enough to the best previous models to justify more tuning.

Experiment 20: MPNet Embeddings + TF-IDF

In this experiment, we kept the strong TF-IDF hazard model from the previous experiments and focused on improving the product-category model. Instead of replacing TF-IDF, we combined it with MPNet sentence embeddings.

For the product model, we used a hybrid representation: **TF-IDF features + MPNet embeddings + hazard-token**. The TF-IDF part captured important words, n-grams, and character patterns, while the MPNet embeddings added semantic information about the whole text. The hazard-token was added to give the product classifier extra information related to the predicted hazard.

The product classifier was LinearSVC with $C = 4.0$ and $\text{emb_weight} = 1.0$.

The results:

METRIC	SCORE
F1 hazard	0.8582
F1 product when hazard correct	0.7669
Official score	0.8126
Kaggle score	0.77362

This experiment achieved a strong validation official score, especially because the product conditional F1 improved.

However, we noticed a clear difference between the validation official score and the Kaggle score. This made us cautious, because the model may have been overfitting to the validation set and might not generalize as well to the private score.

Experiment 21: all-MiniLM-L6-v2 Embeddings

In this experiment, we tested all-MiniLM-L6-v2 sentence embeddings as a lighter alternative to the MPNet embeddings used in the previous experiment. As in the previous experiments, we kept the strong TF-IDF hazard model, which had already achieved a high hazard F1 score. For the product-category task, we tested representations based on all-

Food Hazard Detection ST1 Challenge

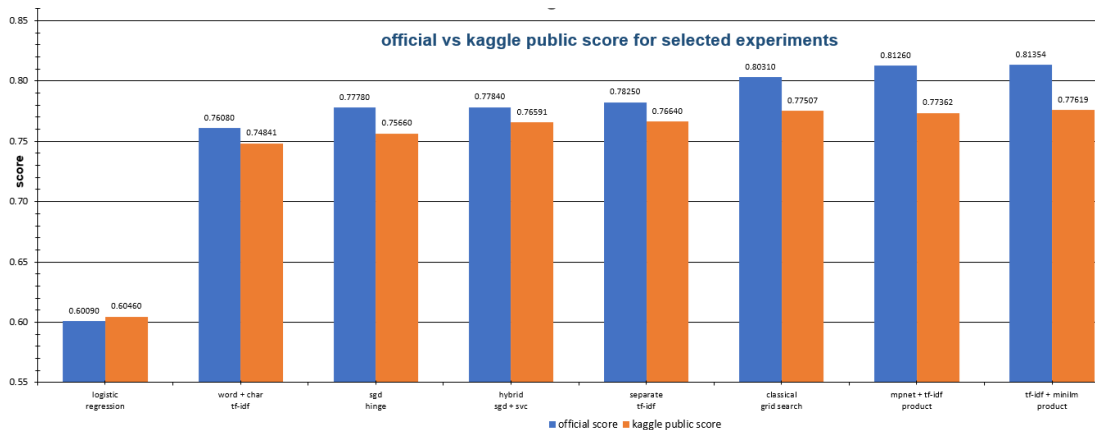
MiniLM-L6-v2 embeddings. Each recall text was converted into a dense 384-dimensional vector, and this representation was used with linear classifiers.

The results:

METRIC	SCORE
F1 hazard	0.8582
F1 product when hazard correct	0.7480
Official score	0.8031
Kaggle score	0.77646

This experiment achieved the **best Kaggle Public score** among our submissions. However, it was not selected as the final best model, because another submitted variation achieved a better Private Kaggle score.

Comparison between the local validation official score and the Kaggle Public score for selected experiments:



From the graph, we can observe that a higher local official score does not always lead to a higher Kaggle Public score. In several experiments, the validation score increased, but the Kaggle Public score stayed almost the same or even decreased slightly.

This suggests that some models may have fitted the validation set better without improving their generalization to the hidden Kaggle Public test set. Therefore, the graph helped us understand that validation performance was useful for comparing experiments locally, but it was not always a perfect indicator of Kaggle performance.

6. Final Selected Model

TF-IDF Hazard Model + TF-IDF/MiniLM Product Model

After Experiment 21, we observed that **all-MiniLM-L6-v2 embeddings** gave a strong Kaggle Public score, but the validation product score was still lower than the MPNet-based experiment. Therefore, instead of using **MiniLM embeddings alone** for the

Food Hazard Detection ST1 Challenge

product-category task, we created a final **hybrid model** that combines the strengths of both **TF-IDF** and **MiniLM embeddings**.

The main change compared to Experiment 21 was in the **product-category model**. In Experiment 21, the product classifier was mainly based on MiniLM sentence embeddings. In the final model, we combined these embeddings with the **TF-IDF product features**. This allowed the model to use both **lexical information** and **semantic information**.

The final system consisted of two separate models:

Label	Final model
hazard-category	TF-IDF word/character features + SGDClassifier
product-category	TF-IDF word/character features + MiniLM embeddings + LinearSVC

Hazard-Category Model

The **hazard model** was kept from the previous experiments because it had already achieved a strong validation F1 score. For this task, we created an input text using the **title**, the full recall **text**, and metadata such as **country**, **year**, and **month**. The title was repeated two times because it usually contains important information about the hazard.

Then, the text was represented using **TF-IDF word n-grams** and **TF-IDF character n-grams**. The word-level TF-IDF features captured important words and short phrases, such as “undeclared milk”, “foreign body”, and “listeria”. The character-level TF-IDF features captured smaller patterns inside words, which helped with word variations and different endings.

These two TF-IDF representations were combined and given to an **SGDClassifier with hinge loss**. With hinge loss, the classifier behaves similarly to a **linear SVM**. We also used `class_weight="balanced"` to reduce the effect of class imbalance.

Product-Category Model

For the **product-category model**, we used a stronger **hybrid representation**. First, we created TF-IDF product features using the title repeated three times, the recall text, and metadata. The title was repeated three times because the product name is often clearly mentioned in the title, so we wanted to give this information more weight.

The product TF-IDF representation used both **word n-grams** and **character n-grams**. For the word features, we used up to trigrams, because product categories often appear as longer phrases, such as “meat egg dairy” or “cereals bakery products”.

Next, we created **MiniLM sentence embeddings** for the same product texts. For this part, we used the **all-MiniLM-L6-v2 SentenceTransformer** model. Each recall text was converted into a dense **384-dimensional vector**. These embeddings were used to capture semantic information from the whole text.

Unlike the TF-IDF input, we did not add metadata tokens to the MiniLM input. This was because the embedding model works better with natural text than with artificial tokens such as `country_US` or `year_2023`.

Food Hazard Detection ST1 Challenge

Parameter Selection

Before training the final product classifier, we ran a small **validation script** to test different combinations of `embedding_scale` and `C`. The goal was to find the best balance between the TF-IDF features and the MiniLM embeddings.

The best combination found was:

`embedding_scale = 0.5`

`C = 2.0`

The `embedding_scale` controls how much influence the MiniLM embeddings have when they are combined with the TF-IDF features. This is useful because TF-IDF features and dense embedding vectors have different value ranges. The `C` parameter controls the regularization strength of the **LinearSVC** classifier.

Using these selected values, the final product classifier was trained with:

TF-IDF product features + MiniLM embeddings → LinearSVC

The results:

METRIC	SCORE
F1 hazard	0.8582
F1 product when hazard correct	0.7688
Official score	0.8135
Kaggle score	0.77619

Compared to Experiment 21, the **hazard F1 stayed the same**, because the hazard model was not changed. The main improvement was in the **product-category prediction**. The product conditional F1 increased from **0.7480** to **0.7688**, and the official validation score increased from **0.8031** to **0.8135**.

This shows that using **MiniLM embeddings alone** was not the best option for the product-category task. The **hybrid representation** performed better because TF-IDF captured exact words, product names, n-grams, and character patterns, while MiniLM added semantic information about the whole recall text.

Therefore, this experiment became our strongest validation experiment and was kept as a candidate for the final submission. The final decision based on the Kaggle Public and Private scores is discussed separately in the evaluation section.

7. Kaggle Final Evaluation

For the final Kaggle evaluation, we compared the two strongest submissions. The first one was the **all-MiniLM-L6-v2 experiment**, which achieved the best Kaggle Public score. The second one was the **hybrid TF-IDF + MiniLM product model**, which was the final selected model.

Food Hazard Detection ST1 Challenge

Model	Submission file	Official score	Public score	Private score
all-MiniLM-L6-v2 Embeddings	all-miniM-L6-v2-submission.csv	0.8031	0.77646	0.76522
TF-IDF Hazard + TF-IDF/MiniLM Product	submission_tfidf_minilm_product.csv	0.8135	0.77619	0.76871

Although Experiment 21 had a slightly better **Public score** than the final model, the final model achieved a better **Private score**. Since the Private leaderboard is the final evaluation used by Kaggle, we selected the **TF-IDF Hazard + TF-IDF/MiniLM Product** model as our final submitted system.

Validation Score vs Kaggle Score

An interesting observation was that many of our experiments had a noticeable difference between the **local validation official score** and the **Kaggle score**. For example, the final model achieved a validation official score of **0.8135**, while its Kaggle Public score was **0.77619** and its Private score was **0.76871**.

This difference suggests that the validation set was not perfectly representative of the hidden Kaggle test data. It also indicates that some experiments may have slightly overfitted to the validation set, even when they performed well locally.

Public and Private Leaderboard Ranking

Another important point is the difference between the **Public** and **Private** leaderboard rankings. During the challenge, our team reached around position **18** on the Public leaderboard, and by the end of the public phase we finished around position **25**. However, after the final Private evaluation, we finished around position **54**.

The score difference between Public and Private was:

Public score: 0.77619

Private score: 0.76871

Difference: 0.00748

This shows that the numerical difference between the two scores was relatively small. However, the ranking changed more significantly, because the final position also depends on how the other teams performed on the Private set. Therefore, even a small difference in score can lead to a larger difference in ranking when many teams have close results.

Summary

Overall, the final model was selected because it achieved the best **Private score** among our submissions. This also showed that relying only on the Public leaderboard can be misleading, because the Public score does not always reflect the final Private performance.

8. Limitations

The main limitation was the strong **class imbalance** in the dataset, since some hazard and product categories had very few examples. This made rare classes harder to predict.

Also, the **validation score** did not always match the **Kaggle score**, which suggests that the validation set was not always fully representative of the hidden test data.

Finally, the **product-category** task was more difficult than the hazard-category task, because product classes were more detailed and sometimes similar to each other.

9. Computational Setup

The experiments were implemented in Python using Google Colab, with the data stored in Google Drive.

The classical TF-IDF models were lightweight and ran fast on CPU. For heavier experiments, such as MiniLM, MPNet, and RoBERTa/BERT-based models, we used CUDA when available, so the code could use the Colab T4 GPU instead of only the CPU. This mainly helped with transformer training and MiniLM embedding creation.